```
MEMory:READ? 0,20000000,16
```

RDT

User manual

Read, watch, chart and change firmware variables and memory over the device's own COM port or LAN link, with no debug probe.

Remote Debugger Tool · Windows app and command line · one ELF per MCU · SCPI over COM, TCP or UDP

WHAT RDT DOES

From a variable’s name to its live value, over the product’s own link

Your link

A COM port, or LAN with TCP or UDP. No JTAG or SWD probe.

Your commands

RDT speaks the SCPI read and write commands you define.

Your ELF

Each MCU’s ELF turns a name into an address and a type.

TWO WAYS TO OPERATE IT

One workspace, two front ends

`rdt-gui`

The GUI

Watch tabs, a live chart, a memory view and an SCPI console. Part 2.

`rdt`

The command line

Every device operation as a one-shot command, for scripts and quick checks. Part 3.

Both read and write the same workspace files.

PART 1

Before you start

Install · Try it on the simulator · What you need to know

INSTALL

One setup, no admin rights, nothing else to install

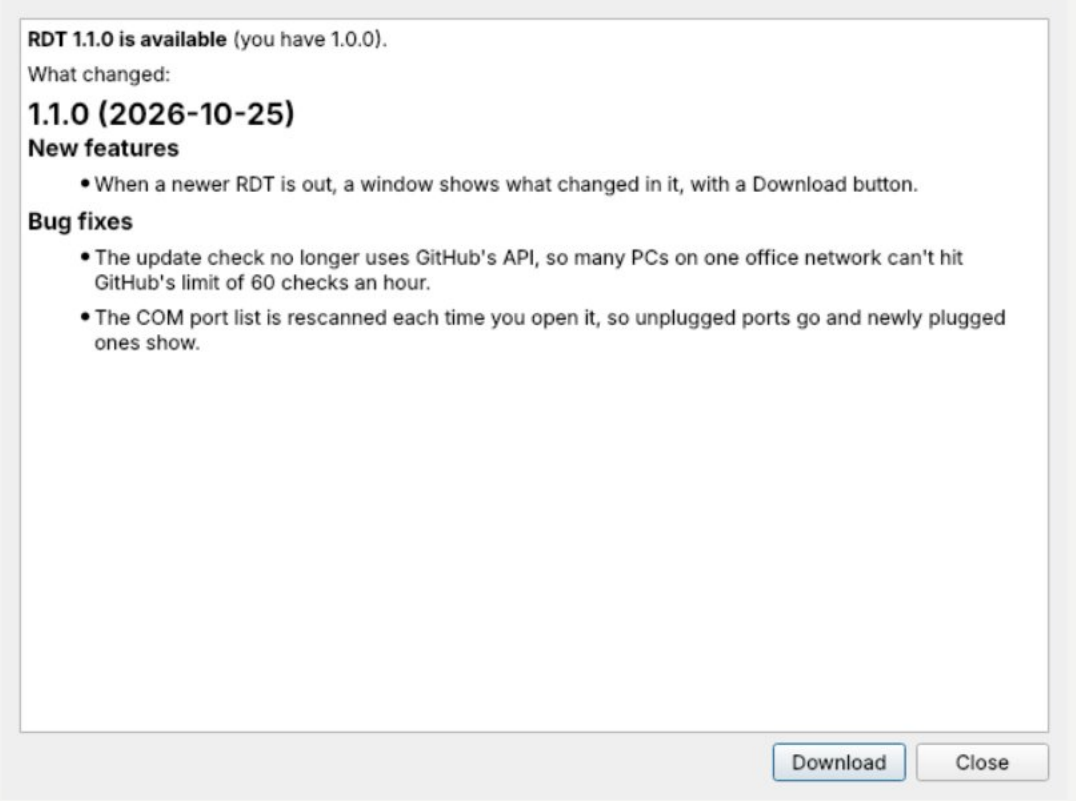
- Run **RDT-Setup-<version>.exe** from the GitHub releases page
- Installs for your Windows user only, with Python and every package inside
- Offers **Add rdt to PATH**; .rdtw files open in RDT on a double-click
- Or unzip the portable zip anywhere and run rdt-gui.exe
- Uninstall asks before deleting your settings

Program	What it is
`rdt-gui`	The window (Start menu: RDT)
`rdt`	The command line
`rdt-sim`	The simulated device

UPGRADE

RDT tells you when a new version is out

- Checks GitHub once a day, with no sign-in
- Shows what changed, with a Download button
- Run the newer setup: settings and workspaces are kept
- **Help > Release notes** lists every version



TRY IT ON THE SIMULATOR

A simulated device stands in for the firmware

Two MCUs, **main** and **motorctl**, each with an example ELF, on TCP and UDP port 5025.

1. Start the simulator
2. Open the example workspace
3. Press Connect (F5): the Motor tab starts to move

```
# terminal 1
python -m sim
# terminal 2
rdt-gui examples/sim_bench.rdtw
# or from the command line
rdt -w examples/sim_bench.rdtw watch
```

WHAT YOU NEED TO KNOW

A workspace keeps everything about one bench

File	Holds
`bench.rdtw`	Connection, MCUs and ELF files, polling, read-only default
`bench.profile.toml`	The SCPI profile
`bench.memory.toml`	The memory map: ranges besides the ELF sections
`bench.watch.toml`	The watch tabs and the chart
`bench.commands.toml`	The device’s known SCPI commands, for the console

The GUI and the CLI read and write the same files.

Every name starts with its MCU

Expression	Meaning
`main:tick_count`	A global variable
`main:motor.speed`	A struct member, to any depth
`main:gains[2]`	An array cell
`main:head→next→value`	Following a pointer
`main:motor.c:state`	A file-static variable, with its source file
`@main:0x20000010:u32`	A raw address as a basic type

MCU 0 is the main MCU: it owns the link and passes requests on to the others. Each MCU has its own ELF.

SCPI PROFILE

You tell RDT how your firmware spells read and write

```
[scpi]
read  = "SERV:MEM:READ? {mcu},{addr},{len}"
write = "SERV:MEM:WRIT {mcu},{addr},{value},{size}"
align        = 4
max_response = 1024
timeout_ms   = 500
```

- Addresses are 8-digit bare hex: **08000000**
- Replies are comma-separated 32-bit hex words
- A failed command gets no reply, so it times out
- Every write is confirmed by reading it back
- RDT never sends *IDN? or SYST:ERR? by itself

SAFETY BUILT IN

RDT only touches memory it knows is safe

Memory map

Access outside the ELF sections and your listed ranges is refused before anything is sent. Register blocks are read alone, at their exact size. Code and constants are read-only.

ELF check

On connect, RDT compares a sample of flash with each ELF. On a mismatch, every read and write to that MCU is refused, so a stale ELF never hits the wrong address.

PART 2

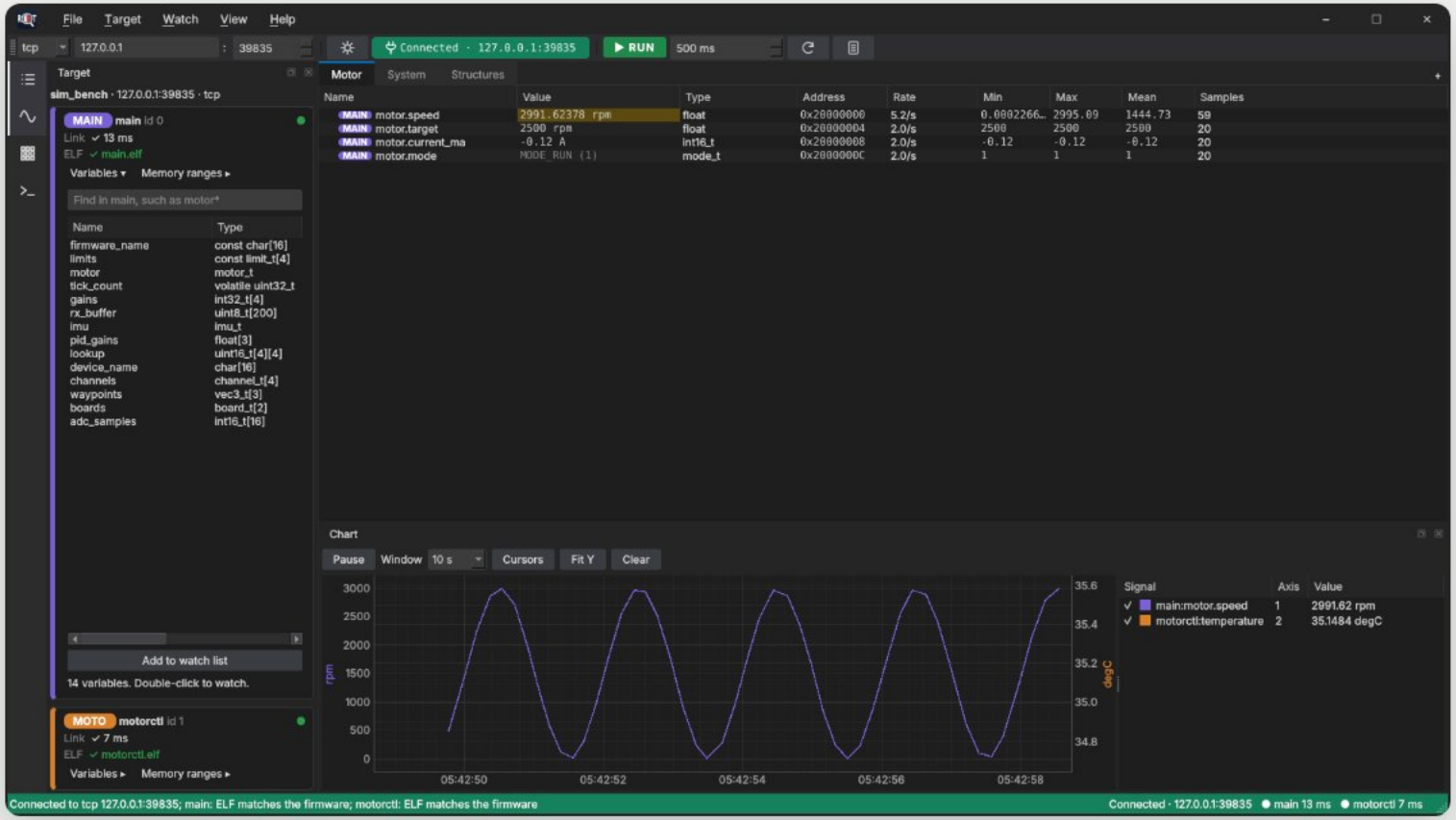
Operating from the GUI

Set up · Connect · Watch · Write · Chart · Memory · Console

THE MAIN WINDOW

Everything on one screen

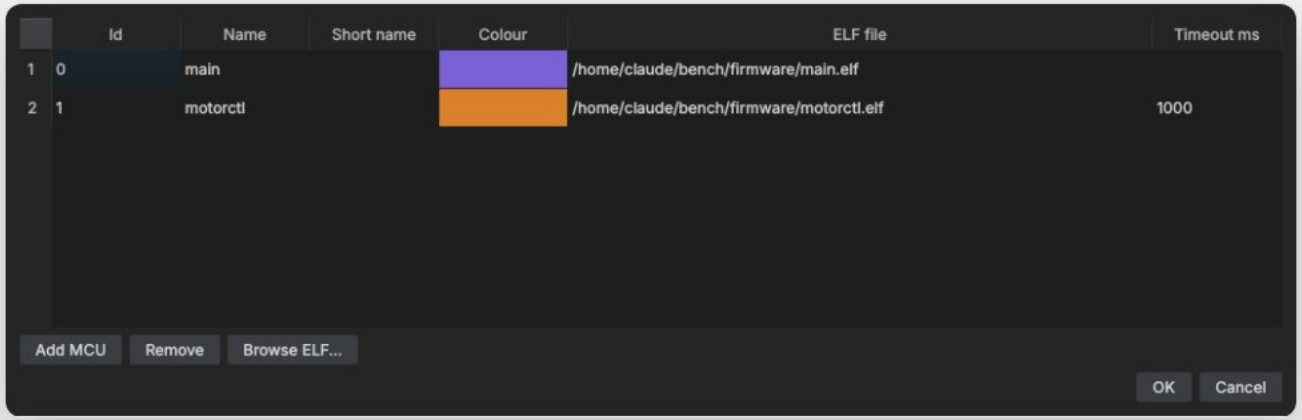
1. Menus and toolbar: link, profile, Connect, RUN/STOP
2. Error bar, only when there is an error
3. Target panel: one card per MCU
4. Watch tabs in the centre
5. Chart, Console and Memory below
6. Status bar with each MCU’s reply time



SET UP A BENCH

Once per bench, then save

1. File > New workspace, or open one
2. Pick the link: serial, tcp or udp
3. Set the SCPI profile (gear icon)
4. Add the MCUs and their ELF files
5. Import the memory map
6. Import known commands (optional)
7. File > Save as

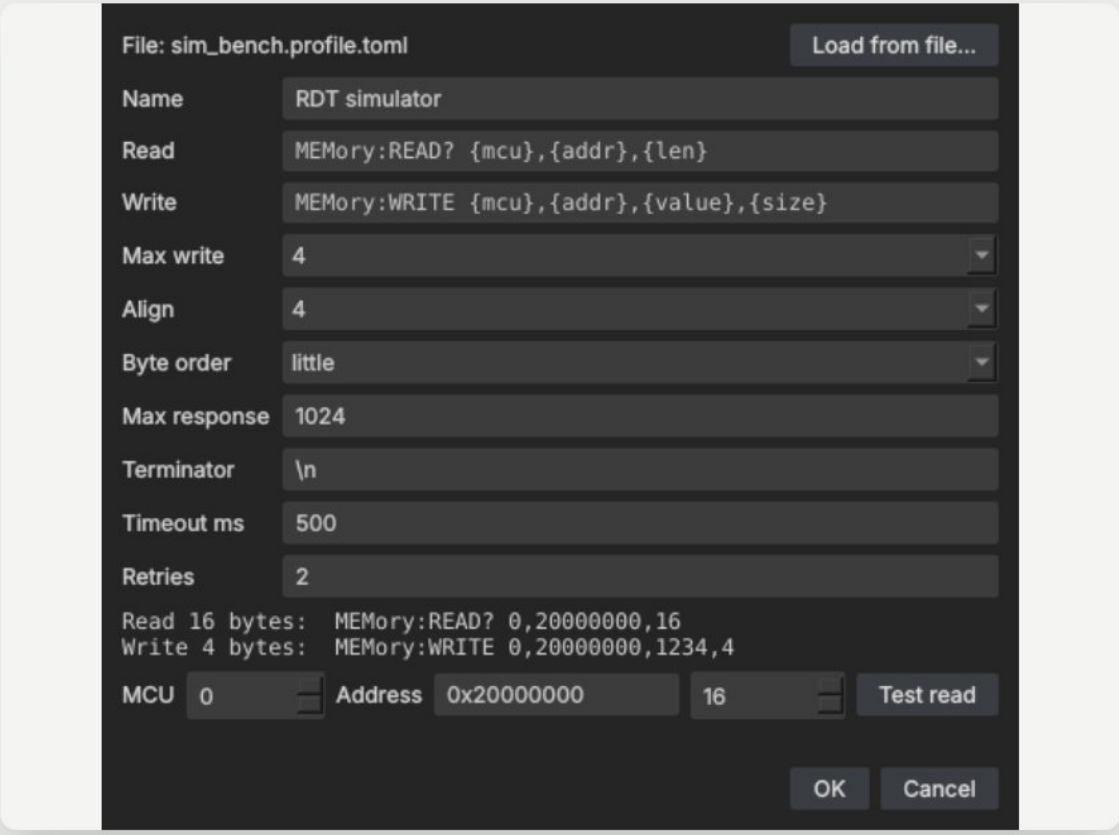


Target > MCUs and ELF files: id, name, short name, colour, ELF and timeout

SCPI PROFILE EDITOR

See the exact commands before you apply

- Read and write templates, alignment, timeouts
- Shows each command as it will be sent
- Refuses a template that can’t work
- **Test read** tries the edited profile on the device
- **Load from file** fills the fields from a profile



CONNECT

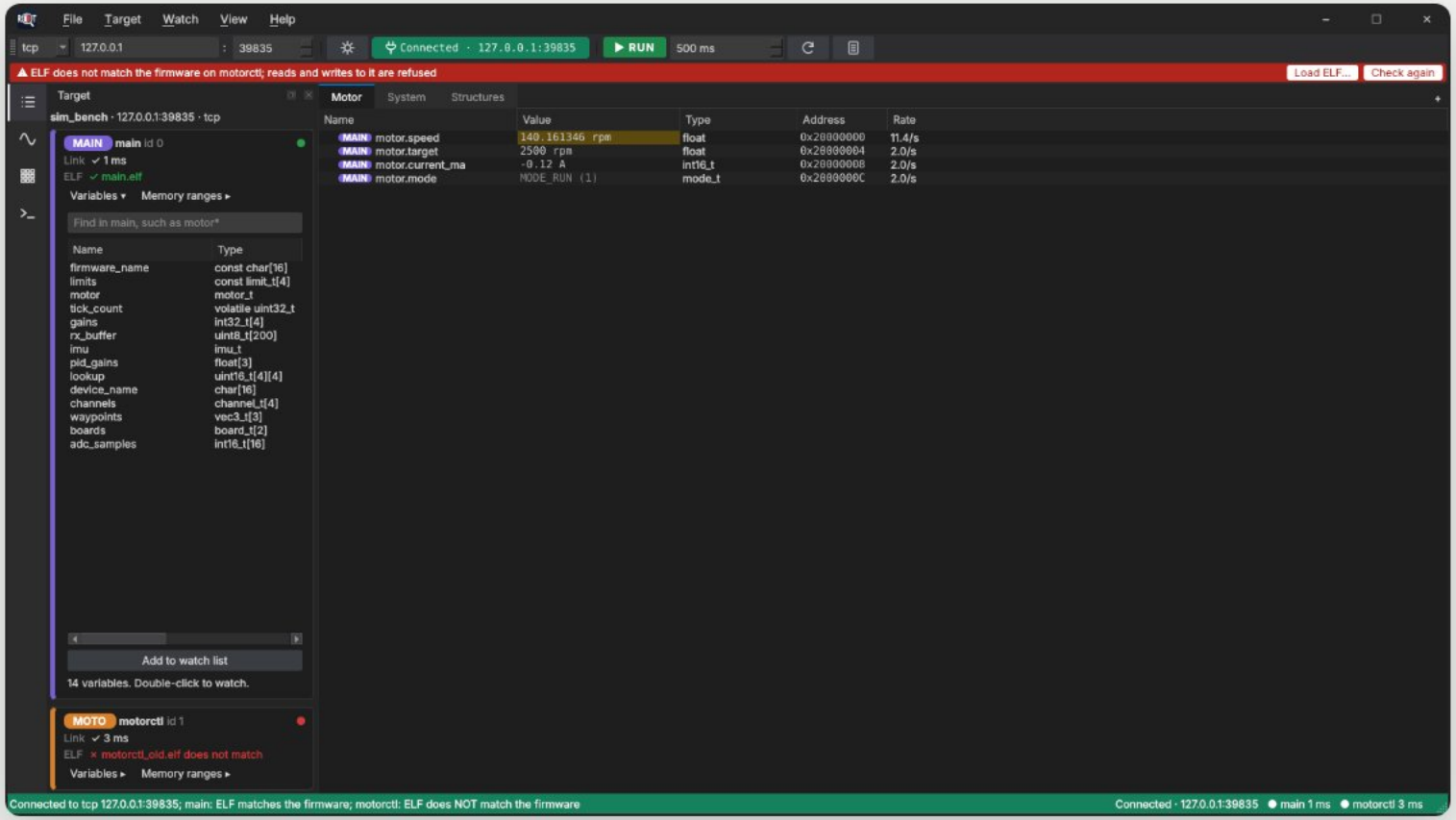
One button shows the link and toggles it (F5)

Disconnected · COM3	Not connected. A click connects.
Connecting · COM3	Opening the port or socket. A click cancels.
Connected · COM3	The port or socket is open. A click disconnects.
Link down · COM3	Port broke, or three reads got no reply. Writes refused; RDT reconnects.

MCU CARDS

Each MCU’s health at a glance

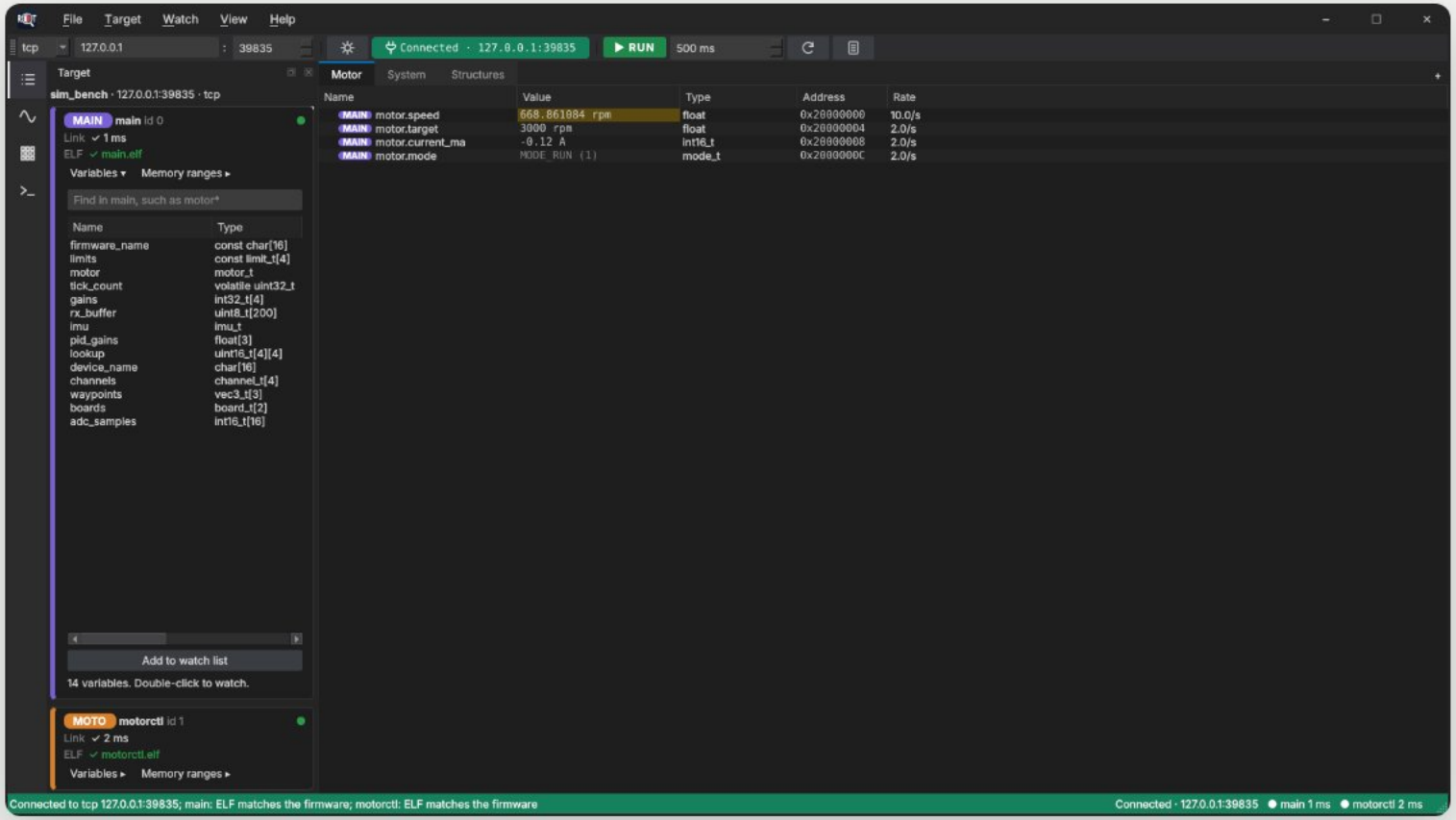
- Dot: green OK, amber slow, red mismatch or link down
- Link: reply time, or silent
- ELF: matches, does not match, or not checked
- On a mismatch, the red bar offers Load ELF and Check again



WATCH VARIABLES

Live values by name

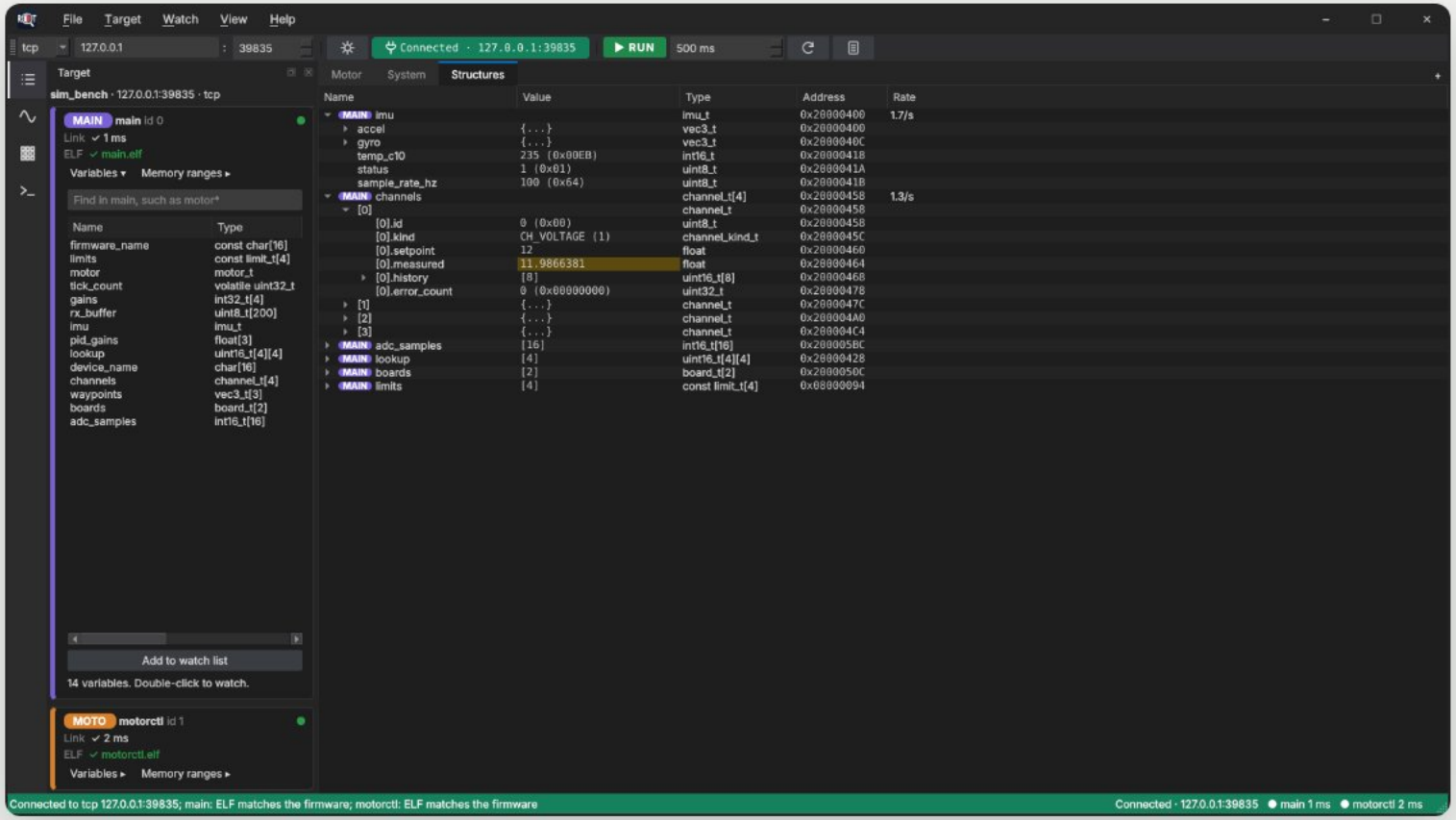
- Open an MCU card to search its globals
- Double-click, Add to watch list, or drag
- Ctrl+E adds any expression
- Each row: MCU tag, value, type, address, rate
- A changed value flashes yellow



STRUCTS AND ARRAYS

Open any struct or array to every field

- Collapsed items are not read
- Arrays show 100 cells at a time
- Enums as names, char arrays as text
- Follow a pointer, or watch one field alone
- Several named watch tabs, all polling

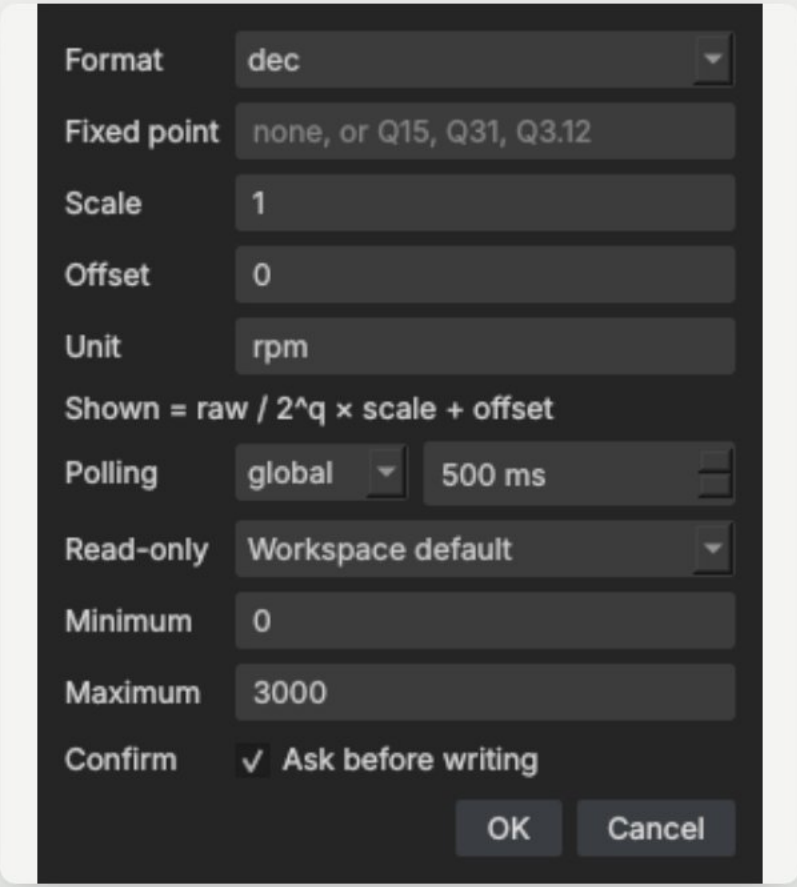


ITEM SETTINGS

Show raw values in real units

- Format: dec, hex, bin or char
- Fixed point (Q15, Q31, Q3.12), scale, offset and unit
- Polling: global, manual, or its own time
- Write safety: read-only, minimum, maximum, ask first

```
shown = raw / 2^q × scale + offset
```

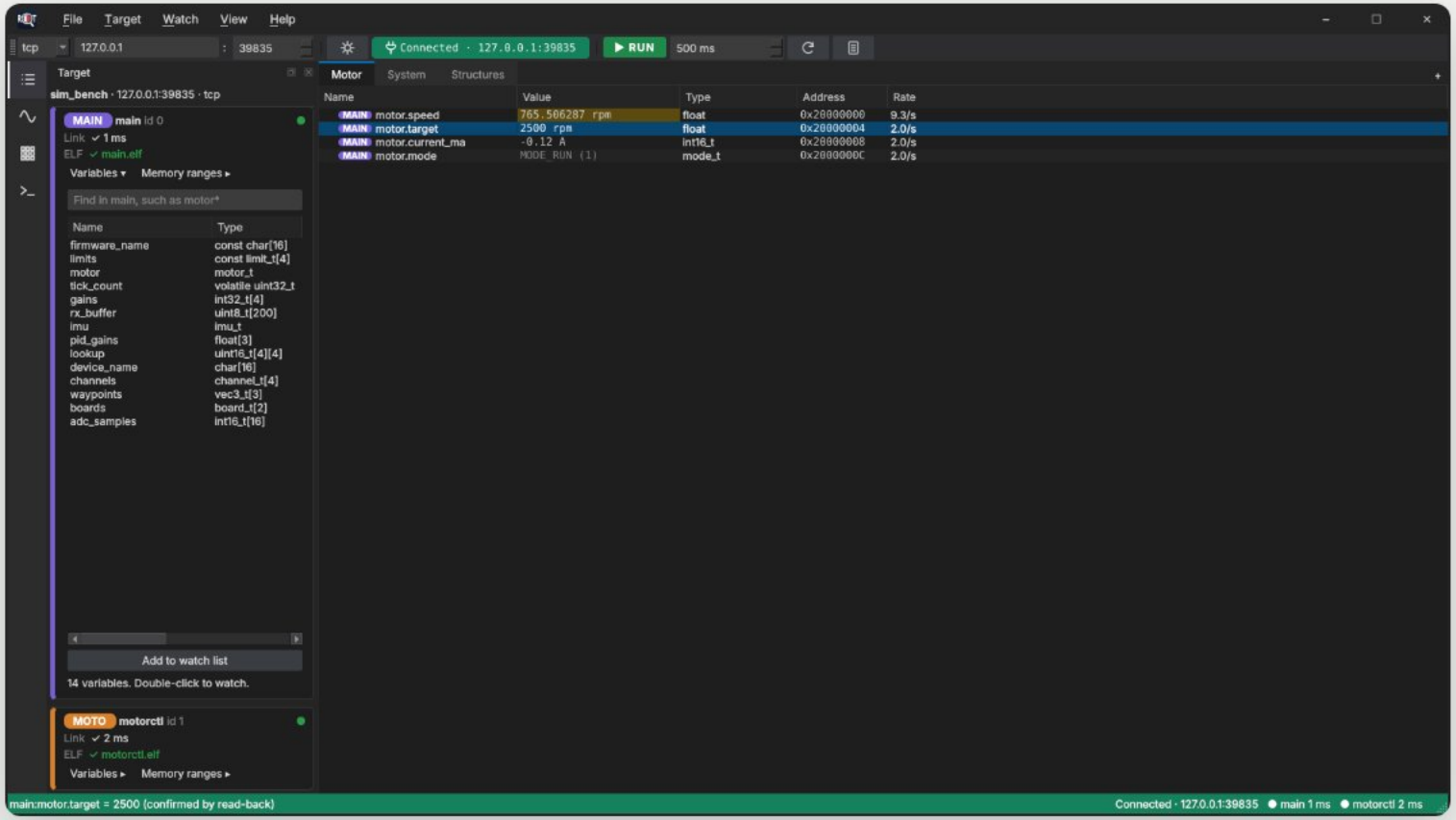


CHANGE A VALUE

Edit in place, confirmed by read-back

1. Double-click the value
2. Type it in the item's units, or an enum name
3. Press Enter: RDT writes and reads it back

Refused when read-only, out of limits, or the ELF does not match.



POLLING

You decide how often each value is read

▶ RUN

RUN / STOP (F7)

Stops or starts all polling. The global interval sits next to it, 500 ms by default.

Poll now (F6)

Reads every watched item once, in every tab, whether polling runs or not.

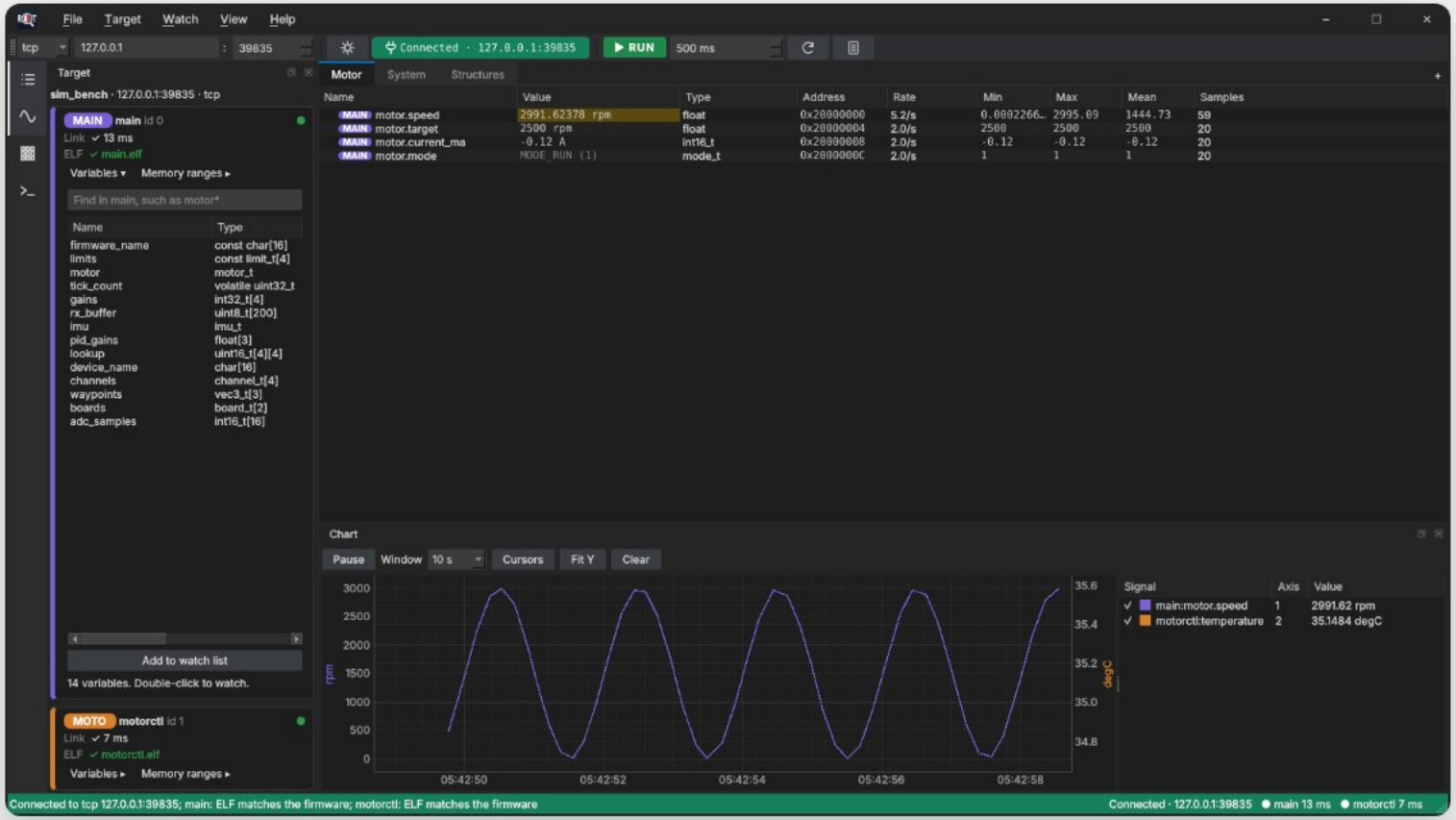
Merged reads

Nearby items on one MCU come in one command, from the same moment.

CHART, STATISTICS, CSV

A live scope for firmware values

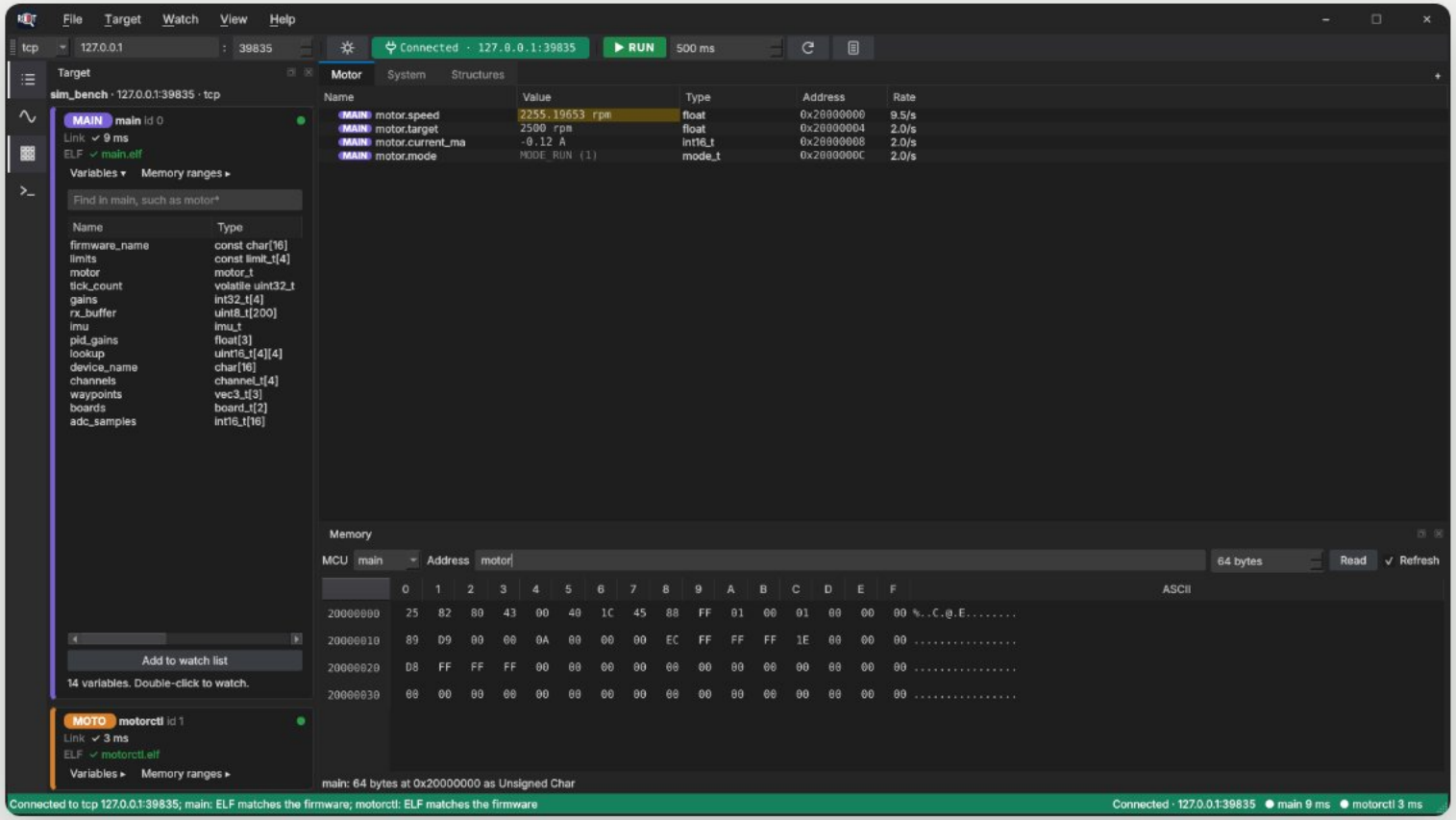
- Ctrl+G, the wave button, or drag to chart
- Each trace in its MCU's colour
- Up to four Y axes, by unit
- Pause, zoom, Fit Y, two cursors
- Min, Max, Mean, Samples (Ctrl+T)
- Log CSV (Ctrl+L), opens in Excel



MEMORY VIEW

Raw memory, in the format you need

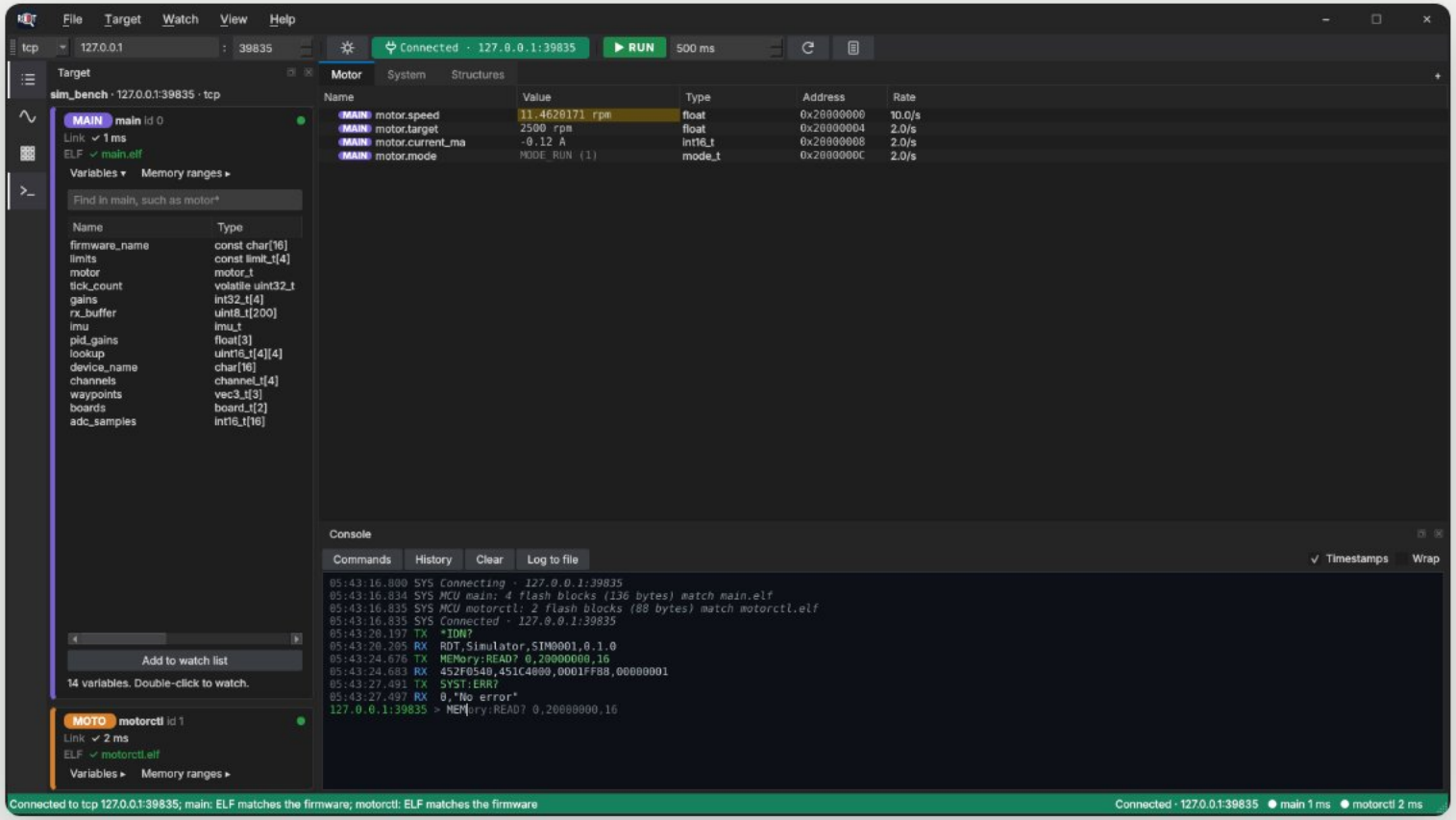
- Pick the MCU, an address or a variable, a length
- Refresh reads again at the polling interval
- Right-click: signed or unsigned, 8 to 32 bits, ASCII, float, double
- Double-click a cell to write it



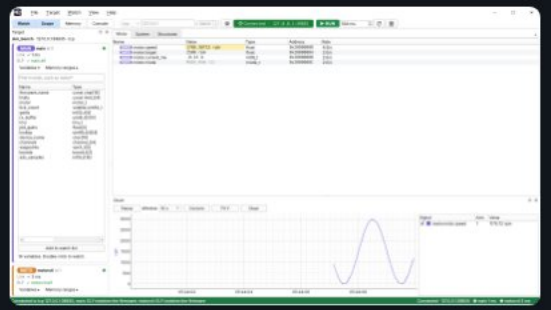
SCPI CONSOLE

A terminal to the firmware

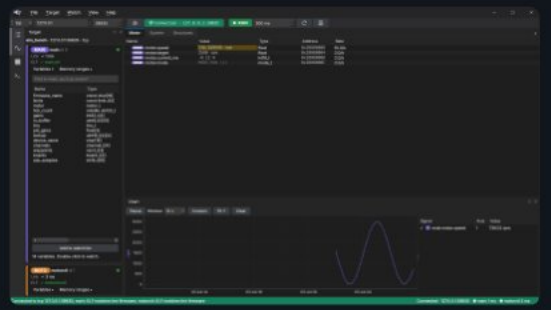
- TX, RX, ERR and SYS lines with times
- Suggestions from history and known commands
- Commands list by group, with a filter
- Find, log to file, send break, hex and text
- Works while polling runs



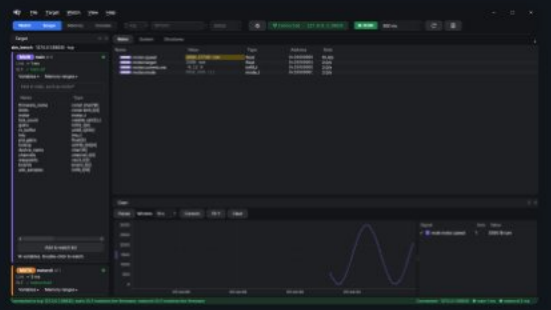
Make it yours: panels, themes, density



Light



Studio dark



Islands



Instrument

- Ctrl+1 to Ctrl+4 show or hide Watch, Scope, Memory, Console; drag panels anywhere
- View > Theme: System, Light, Dark, Studio dark, Islands, Instrument, Light pro
- View > Density: compact or comfortable; all kept per user in ui.toml

SAVE, IMPORT AND EXPORT

Move one part at a time between workspaces

Watch list

The watch tabs and the chart; add or replace

Commands

The device’s known commands, as .toml or CSV

Memory map

All memory ranges; RDT reconnects to use them

Interface settings

Theme, density, console and memory view

File > Save (Ctrl+S) writes the .rdtw and its files side by side; the title shows * while unsaved.

KEYBOARD SHORTCUTS

The keys worth learning (F1 lists all)

Keys	Action
F5	Connect or disconnect
F6	Poll now
F7	RUN / STOP polling
Ctrl+E	Add expression
Ctrl+G	Add selected to chart
Ctrl+L	Log to CSV

Keys	Action
Ctrl+T	Statistics columns
Ctrl+1 to 4	Show or hide panels
Ctrl+Shift+T	New watch tab
Ctrl+Shift+S	Save layout
Space	Pause or run the chart
Ctrl+F, Ctrl+K	Find, clear (console)

PART 3

Operating from the command line

`rdt info` · `read` · `write` · `dump` · `watch` · `send` · `console`

CHOOSE THE TARGET

A workspace, or rdt.toml, plus overrides

```
rdt -w bench.rdtw info   # a workspace
rdt info                 # rdt.toml here, else in %LOCALAPPDATA%\rdt
```

Option	Meaning
`--serial COM5 --baud 115200`	Use a COM port
`--tcp HOST:PORT, --udp`	Use LAN
`--profile, --timeout, --mcu`	Override for this run only
`--allow-mismatch`	Use an MCU whose ELF does not match
`-v, -vv`	Log link events; every command and reply

LOOK AT THE DEVICE

rdt info: the link, each ELF check, the memory map

```
$ rdt -w bench.rdtw info
Link:          tcp 127.0.0.1:5025
Profile:       RDT simulator
MCU 0 main (main.elf, little endian)
    identity: match, 4 flash blocks (136 bytes) match main.elf
    0x08000000     80  ro  section     .text
    0x20000000     36  rw  section     .data
    0x48000000   1024  rw  peripheral  GPIOA
MCU 1 motorctl (motorctl.elf, little endian)
    identity: match, 2 flash blocks (88 bytes) match motorctl.elf
```

rdt symbols main 'motor*' finds variables and **rdt ptype main:motor** shows a type's layout, with no device needed.

READ AND WRITE

Every line of read can be pasted into write

```
$ rdt -w bench.rdtw read main:motor
main:motor  motor_t, 16 bytes at 0x20000000
main:motor.speed      = 27.0084591     float
main:motor.target     = 3000           float
main:motor.current_ma = -120 (0xFF88)  int16_t
main:motor.mode       = MODE_RUN (1)   mode_t
$ rdt -w bench.rdtw write main:motor.target 2500 -y
main:motor.target = 2500 rpm (confirmed by read-back)
$ rdt -w bench.rdtw dump main 0x20000000 32
20000000  D9 37 AD 44 00 40 1C 45 88 FF 01 00 01 00 00 00  |.7.D.@.E........|
```

Also **poke** for raw bytes. Item scaling and write safety apply on the command line too.

WATCH, LOG AND SEND

Stream values, log to CSV, talk SCPI

```
$ rdt -w bench.rdtw watch main:motor.speed motorctl:temperature -i 200
time          main:motor.speed  motorctl:temperature
05:45:33.535    2418.24316 rpm       36.8205338 degC
05:45:33.638    2747.80127 rpm       36.8205338 degC
05:45:33.739    2944.02393 rpm       36.8205338 degC
$ rdt -w bench.rdtw watch --tab Motor --csv run1.csv
$ rdt -w bench.rdtw send "*IDN?"
RDT,Simulator,SIM0001,0.1.0
$ rdt -w bench.rdtw console
```

watch rides through a lost link and an ELF rebuild. In **rdt console**, lines starting with a dot are RDT's own: .read, .write, .history, .quit.

EDIT A WORKSPACE

Build a bench from a script, no device needed

```
rdt --serial COM3 --profile fw.toml --mcu main=0:main.elf init bench.rdtw
rdt -w bench.rdtw add main:motor.speed --tab Motor --unit rpm --poll 100
rdt -w bench.rdtw add main:motor.target --min 0 --max 3000 --confirm
rdt -w bench.rdtw add main:motor.mode --read-only
rdt -w bench.rdtw tab add Tuning
rdt -w bench.rdtw memory import h7.memory.toml
rdt -w bench.rdtw commands import gac.csv
```

The same import and export as the GUI: memory, watchlist, commands. Plus **rdt ui show** and **rdt history**.

EXIT CODES AND SCRIPTS

Ready for scripts and test benches

Code	Meaning
0	Success
1	Anything failed; stderr says what
11	send --expect got no reply, or no match

```
# the Python API under the CLI and GUI
from rdt import Target
with Target.from_file("bench.rdtw") as t:
  print(t.read_value("main:motor.speed"))
  t.write_value("main:motor.mode",
                "MODE_RUN")
```

COMMAND SUMMARY

Every command at a glance

Command	What it does	Device
`info`	Link, profile, ELF checks, memory map	Yes
`symbols, ptype`	Find variables, show a type's layout	No
`read, write`	Read values; write one and confirm it	Yes
`dump, poke`	Hex dump; write raw bytes	Yes
`watch`	Poll and print, optionally to CSV	Yes
`send, console`	One SCPI command; interactive console	Yes
`init, add, remove, tab`	Create and edit a workspace	No
`memory, watchlist, commands`	Import and export parts	No

PART 4

Troubleshooting

What you see, why, and what to do. Turn on -vv to see every command and reply.

When the device doesn’t answer

What you see	Why	What to do
Every read times out	The read command or link settings are wrong	Check the template and terminator; try Test read; type SYST:ERR?
Link down, port still open	No reply to the last three reads	Check the board runs; reset it if it hung
Link down after unplugging	The COM port disappeared	Plug it back; RDT reconnects by itself
MCU silent	A secondary MCU stopped answering	Check that MCU; RDT retries every 2 s
A reply is refused	Not comma-separated hex words	Check the reply format and align

TROUBLESHOOTING · REFUSED ACCESS

When RDT refuses a read or write

What you see	Why	What to do
ELF does not match the firmware	The device runs another build	Load the matching ELF or flash the matching firmware, then Check again
Outside the MCU’s known memory	No ELF section or listed range	Add the bank or register block to the memory map
Read-only	The item, workspace default or section	Change the item’s settings, or turn off All items read-only
Not confirmed: wrote, read back	The value read back differs	The firmware changed or refused it; check SYST:ERR?

Add only regions the firmware can read safely, and keep register blocks as peripherals.

PART 5

Files and where they are

FILES AND WHERE THEY ARE

Settings live in %LOCALAPPDATA%\rdt

File	Where	What
bench.rdtw and its .toml files	Side by side, default %LOCALAPPDATA%\rdt	The workspace
rdt.toml	Current folder, else %LOCALAPPDATA%\rdt	Default target when none is given
ui.toml	%LOCALAPPDATA%\rdt	GUI settings and layout, per user
console_history.txt	%LOCALAPPDATA%\rdt	Console history, GUI and CLI
cache	%LOCALAPPDATA%\rdt\cache	Parsed ELFs; safe to delete
logs\rdt.log	%LOCALAPPDATA%\rdt\logs	Warnings and errors; safe to delete
The installed RDT	%LOCALAPPDATA%\Programs\RDT	Programs, examples, simulator

WHERE TO READ MORE

Start on the simulator, then point RDT at your board

`docs/USER_MANUAL.md` `docs/GUI.md` `docs/CLI.md` `docs/SIMULATOR.md`

`docs/use-your-own-elf.md`

github.com/rani-feldman/rdt · Help > User manual in the app